# **Project Title:** Anti-Performance Throttling Booster for Applications and Gaming Systems

**Project ID:** 50

**Team Name:** Infinite Loop

## SOFTWARE SPECIFICATION REQUIREMENTS

### Team Details

| Role | Team Member | SRN | Primary Functional Ownership |
|---|---|---|---|
| Member 1 | Ankita Vellara | PES2UG24AM021 | Real-Time Performance Monitoring (REQ-01 to REQ-04) |
| Member 2 | Ashritha Reddy | PES2UG24AM071 | Throttling Detection, Alerts and Recommendations (REQ-05 to REQ-08, REQ-20 to REQ-21) |
| Member 3 | Harshitha Vangaveti | PES2UG24AM064 | Background Process Optimization and Performance Profiles (REQ-09 to REQ-12, REQ-16 to REQ-19) |
| Member 4 | Hamsa M N | PES2UG24AM060 | Network Usage Monitoring and Event/Action Logging (REQ-13 to REQ-15, REQ-22 to REQ-23) |

### Revision History

| Version | Date | Prepared by | Reason for Change |
|---|---|---|---|
| 1.0 | 06-09-2026 | Hamsa M N | Initial SRS. |
| 2.0 | 03-11-2026 | Ankita Vellara | Updated functional and non-functional requirement ownership for the four-member implementation plan; clarified module responsibilities and cross-module coordination. Corrected inconsistencies. |

## Table of Contents

1. Introduction
2. Overall Description
3. External Interface Requirements
4. Analysis Models
5. System Features and Functional Requirements
6. Other Nonfunctional Requirements
7. Other Requirements
Appendix A: Glossary
Appendix B: Field Layouts
Appendix C: Requirement Traceability Matrix
Appendix D: Team Responsibility Matrix

# 1. Introduction

### 1.1 Purpose

The purpose of this software is to develop an Anti-Performance Throttling Booster for Applications and Gaming Systems that detects and helps mitigate performance throttling in laptops and desktop computers. The system monitors CPU and memory utilization, background processes, and network usage to identify conditions that may cause performance degradation. It provides real-time performance monitoring, throttling detection, optimization recommendations, and user-controlled performance profiles to help users maintain smoother application and gaming experiences.

### 1.2 Intended Audience

This software is primarily intended for gamers and users who rely on their laptops or desktop computers for high-performance activities such as gaming, software development, video editing, graphic design, and other resource-intensive applications. It is designed for users who experience performance slowdowns, system resource contention, or throttling while running demanding workloads. The tool helps these users monitor system performance, identify potential causes of slowdowns, and apply suitable optimization measures to improve their overall experience.

### 1.3 Product Scope

The product is a desktop-oriented software tool that detects and helps mitigate performance throttling in applications and gaming systems by monitoring system resources, background processes, and network usage. It provides real-time monitoring and user-controlled performance profiles.

The system will enable users to:

- Monitor CPU, memory, process, and relevant network usage in real time.
- Detects conditions associated with application or game performance throttling.
- Identify background processes and resource usage that may affect the target workload.

- View alerts, detected conditions, and optimization recommendations.
- Select or configure user-controlled performance profiles.
- Apply controlled optimization actions to eligible background processes/resources.
- View monitoring status and maintain logs of detected events and actions.

### 1.4 References

- JACKFRUIT Phase-1 Software Requirements Specification template supplied for UE24CS341A – Software Engineering.
- JACKFRUIT Project 50 scope: Anti-Performance Throttling Booster for Applications and Gaming Systems.
- JACKFRUIT Software Engineering Project Guidelines 2026 and Deliverables Part-2.
- Project GitHub repository and project artifacts maintained by the team.
- Version 1.0 SRS dated 06 September 2026.

## 2. Overall Description

### 2.1 Product Perspective

The product is a standalone desktop-oriented monitoring and optimization tool. It interacts with the host operating system to collect performance information and, where permitted, perform controlled optimization actions. It complements rather than replaces the operating system, applications, or games.

### 2.2 Product Functions

- Real-time performance monitoring.
- Performance-throttling detection.
- Background-process analysis and controlled optimization.
- Network-usage monitoring.
- User profile creation, selection, and modification.
- Alerts and optimization recommendations.
- Event and action logging.

### 2.3 User Classes and Characteristics

| User Class | Characteristics | Main Functions |
|---|---|---|
| End User / Gamer | Basic to intermediate computer knowledge. | Monitor performance, select profiles, view alerts, and apply permitted optimization. |
| Developer / Project Team | Technical knowledge of software development and operating systems. | Configure, implement, test, maintain, integrate, and validate the system. |
| Faculty Evaluator / Tester | Technical reviewer. | Review requirements, execute/review tests, and inspect implementation and results. |

### 2.4 Operating Environment

- Desktop/laptop computer capable of running the target application or game.
- Windows-based environment is the primary development and demonstration target.
- Python and supporting libraries may be used for monitoring and application logic.
- Git and GitHub are used for source control and collaboration.

### 2.5 Design and Implementation Constraints

- Agile lifecycle shall be followed as required by the course.
- GitHub shall be used for version control, collaboration, backlog management, and traceable individual contributions.
- CI/CD, code review, testing, and security validation shall be incorporated into the development process.
- The system shall not terminate or modify critical system processes without safeguards.
- Optimization shall operate only within permissions granted by the operating system/user.
- Monitoring overhead should be kept low so the tool does not itself significantly degrade performance.
- OS-specific operations should be isolated to support possible future portability.

### 2.6 Assumptions and Dependencies

- The operating system exposes sufficient performance and process information.
- The user has required permissions for permitted monitoring and optimization actions.
- The monitored application/game runs on the same host system.
- Effectiveness depends on hardware, OS configuration, workload, and application behavior.
- Third-party libraries and development tools remain compatible with the implementation.
- The modules will be developed separately using agreed data structures/interfaces and integrated through the shared repository.

## 3. External Interface Requirements

### 3.1 User Interfaces

The application shall provide a dashboard-oriented interface showing real-time system status and monitored performance information. It shall provide controls for starting/stopping monitoring, selecting profiles, viewing alerts and logs, and initiating permitted optimization actions. Status messages shall clearly indicate success, warning, or failure.

### 3.2 Software Interfaces

- Operating-system APIs/libraries for CPU, memory, process, and network metrics.
- Process-management interfaces for permitted optimization operations.
- Local storage or lightweight database/file storage for profiles and logs.
- Testing and static-analysis tools used by the development workflow.
- GitHub for source-code version control, collaboration, backlog, and pull-request review.

### 3.3 Communications Interfaces

Core monitoring and optimization are intended to operate locally. If a web/local dashboard is implemented, HTTP/HTTPS may be used for communication. Network monitoring shall collect usage statistics and shall not intercept private application content. Communication shall avoid unnecessary exposure of system information.

## 4. Analysis Models

The initial logical interaction model is: User → Monitoring Engine → Throttling Detection → Cause Analysis → Profile/Optimization Engine → Dashboard/Alerts. Network monitoring collects relevant usage statistics, while the logging component records detected events and optimization actions.

Primary use cases: Start Monitoring, View Metrics, Detect Throttling, Analyze Cause, Select Profile, Apply Optimization, View Alerts/Logs, and Stop Monitoring. A formal UML use-case diagram and detailed architecture diagrams should be added during the design phase and referenced from this section.

### 4.1 Module Boundaries and Integration

| Module | Owner | Inputs / Dependencies | Outputs |
|---|---|---|---|
| Real-Time Performance Monitoring | Member 1 – Ankita Vellara | Host OS performance/process information; user start/stop control. | CPU, memory, process metrics and monitoring status. |
| Throttling Detection, Alerts and Recommendations | Member 2 – Ashritha Reddy | Metric samples from Member 1; configured detection criteria. | Suspected throttling events, alerts and recommendations. |
| Process Optimization and Profiles | Member 3 – Harshitha Vangaveti | Process information from Member 1; profile settings owned by this module; detection/recommendation information where available. | Confirmed optimization actions and action results. |
| Network Monitoring and Logging | Member 4 – Hamsa M N | Host network statistics; event/action records from Members 2 and 3. | Network usage statistics, network-condition alerts, and recent event/action logs. |

The modules may be implemented and unit-tested independently using mock/test data. Before parallel implementation, the team shall agree on shared data structures for performance metrics, process information, detection events, profile settings, and log events. Integration is required to produce the complete working application. Runtime data dependencies do not require members to develop simultaneously.

## 5. System Features and Functional Requirements

Each functional requirement retains its Version 1.0 identifier for traceability. The responsible member is the primary implementation owner. Other members may contribute to integration, review, and testing.

### 5.1 Real-Time Performance Monitoring

Collect and display current performance information while monitoring is active.

Priority: High.

Owner: Member 1 – Ankita Vellara.

Stimulus/Response: User starts monitoring → system initializes collection → system samples metrics → dashboard updates.

| Requirement ID | Functional Requirement | Responsible Member |
|---|---|---|
| REQ-01 | The system shall allow the user to start and stop real-time monitoring. | Member 1 – Ankita Vellara |
| REQ-02 | The system shall display current CPU and memory utilization. | Member 1 – Ankita Vellara |
| REQ-03 | The system shall display relevant process-level resource usage. | Member 1 – Ankita Vellara |
| REQ-04 | The system shall refresh monitoring information automatically at a defined interval. | Member 1 – Ankita Vellara |

### 5.2 Performance Throttling Detection

Detects conditions indicating possible performance degradation or throttling.

Priority: High.

Owner: Member 2 – Ashritha Reddy.

Stimulus/Response: Monitoring data is collected → detection logic evaluates configured criteria → abnormal condition is identified → system generates a throttling event.

| Requirement ID | Functional Requirement | Responsible Member |
|---|---|---|
| REQ-05 | The system shall evaluate monitored metrics against configured detection criteria. | Member 2 – Ashritha Reddy |
| REQ-06 | The system shall identify and report suspected performance-throttling conditions. | Member 2 – Ashritha Reddy |
| REQ-07 | The system shall record the time and relevant metrics associated with a detected event. | Member 2 – Ashritha Reddy |
| REQ-08 | The system shall avoid reporting a condition as certain when the available evidence only indicates a suspected event. | Member 2 – Ashritha Reddy |

### 5.3 Background Process Optimization

Identify resource-intensive background processes and provide controlled optimization actions.

Priority: High.

Owner: Member 3 – Harshitha Vangaveti.

Stimulus/Response: System identifies resource contention → identifies eligible background processes → user reviews recommendation → user confirms action → system applies permitted optimization → result is displayed/logged.

| Requirement ID | Functional Requirement | Responsible Member |
|---|---|---|
| REQ-09 | The system shall list relevant running processes and their resource usage. | Member 3 – Harshitha Vangaveti |
| REQ-10 | The system shall identify eligible background processes that may affect performance. | Member 3 – Harshitha Vangaveti |
| REQ-11 | The system shall require user confirmation before disruptive optimization actions. | Member 3 – Harshitha Vangaveti |
| REQ-12 | The system shall report the result of each optimization action. | Member 3 – Harshitha Vangaveti |

### 5.4 Network Usage Monitoring

Monitor network usage relevant to application/game performance.

Priority: Medium.

Owner: Member 4 – Hamsa M N.

Stimulus/Response: Monitoring is active → system collects network-usage statistics → system displays usage and flags configured high/unusual usage.

| Requirement ID | Functional Requirement | Responsible Member |
|---|---|---|
| REQ-13 | The system shall display relevant network usage statistics. | Member 4 – Hamsa M N |
| REQ-14 | The system shall associate network usage information with the monitoring period. | Member 4 – Hamsa M N |
| REQ-15 | The system shall alert the user when configured network-usage conditions are exceeded. | Member 4 – Hamsa M N |

### 5.5 User-Controlled Performance Profiles

Provide configurable modes for different workloads.

Priority: High.

Owner: Member 3 – Harshitha Vangaveti.

Stimulus/Response: User opens profiles → selects or configures a profile → system loads profile settings → optimization uses those settings.

| Requirement ID | Functional Requirement | Responsible Member |
|---|---|---|
| REQ-16 | The system shall provide predefined performance profiles. | Member 3 – Harshitha Vangaveti |
| REQ-17 | The system shall allow users to select an active profile. | Member 3 – Harshitha Vangaveti |
| REQ-18 | The system shall allow supported profile settings to be modified. | Member 3 – Harshitha Vangaveti |
| REQ-19 | The system shall apply the active profile to relevant optimization decisions. | Member 3 – Harshitha Vangaveti |

### 5.6 Alerts, Recommendations and Logging

Communicate detected issues, recommendations, and actions.

Priority: Medium.

Shared feature ownership: alerts/recommendations by Member 2; event/action logging by Member 4.

Stimulus/Response: System detects an event → generates alert/recommendation → user views it → system records the event/action.

| Requirement ID | Functional Requirement | Responsible Member |
|---|---|---|
| REQ-20 | The system shall display alerts for relevant detected conditions. | Member 2 – Ashritha Reddy |
| REQ-21 | The system shall provide optimization recommendations when sufficient information is available. | Member 2 – Ashritha Reddy |
| REQ-22 | The system shall log detected events and optimization actions with timestamps. | Member 4 – Hamsa M N |
| REQ-23 | The system shall allow the user to view recent logs. | Member 4 – Hamsa M N |

## 6. Other Nonfunctional Requirements

The following ownership assignments identify the primary member responsible for implementing or coordinating each non-functional requirement (NFR). NFRs often apply across multiple modules; ownership does not remove the responsibility of other members to comply.

### 6.1 Performance Requirements

| NFR ID | Requirement | Primary Owner | Supporting Members |
|---|---|---|---|
| NFR-PER-01 | The dashboard should update frequently enough to support real-time monitoring. | Member 1 – Ankita Vellara | Member 2 and Member 4 for alerts/network display. |
| NFR-PER-02 | Monitoring should introduce minimal additional CPU and memory overhead. | Member 1 – Ankita Vellara | All members must avoid unnecessary processing in their modules. |
| NFR-PER-03 | Optimization actions should provide a visible result after completion or failure. | Member 3 – Harshitha Vangaveti | Member 2 for user-facing alerts; Member 4 for action logging. |
| NFR-PER-04 | The system should remain responsive while monitoring is active. | Member 1 – Ankita Vellara | All module owners; integration lead coordinates system-level checks. |

### 6.2 Safety Requirements

| NFR ID | Requirement | Primary Owner | Supporting Members |
|---|---|---|---|
| NFR-SAF-01 | Critical operating-system processes shall not be terminated or modified by automated optimization. | Member 3 – Harshitha Vangaveti | Member 1 may expose process information; all members review safety behavior. |
| NFR-SAF-02 | Potentially disruptive actions shall require user confirmation. | Member 3 – Harshitha Vangaveti | Member 2 provides recommendations/alerts. |
| NFR-SAF-03 | The system shall fail safely when an optimization operation is not permitted. | Member 3 – Harshitha Vangaveti | Member 4 records the outcome; Member 2 may display an alert. |
| NFR-SAF-04 | The application shall clearly communicate errors rather than silently applying unsupported actions. | Member 3 – Harshitha Vangaveti | Member 2 for alerts and Member 4 for logs. |

### 6.3 Security Requirements

| NFR ID | Requirement | Primary Owner | Supporting Members |
|---|---|---|---|
| NFR-SEC-01 | Only permitted local system information shall be collected. | Member 1 – Ankita Vellara | Member 4 for network statistics; all members respect data-minimization. |
| NFR-SEC-02 | Logs shall avoid unnecessary sensitive information. | Member 4 – Hamsa M N | All members must avoid sending sensitive data to the logger. |
| NFR-SEC-03 | Access to privileged optimization operations shall respect operating-system permissions. | Member 3 – Harshitha Vangaveti | Member 1 provides process data; all members respect OS permissions. |
| NFR-SEC-04 | Source code and dependencies shall be checked using appropriate security/static-analysis practices. | Shared responsibility; coordinated by the integration/infrastructure lead. | All four members must resolve findings in their own code. |
| NFR-SEC-05 | The system shall not intercept private application content merely for network monitoring. | Member 4 – Hamsa M N | All members respect the local-statistics-only scope. |

### 6.4 Software Quality Attributes

| Quality Attribute | Requirement | Primary Owner | Supporting Members |
|---|---|---|---|
| Usability | Controls and alerts shall be understandable to a typical end user. | Member 1 – Ankita Vellara (dashboard coordination) | Member 2 – alert wording; Member 3 – profile/action controls; Member 4 – log view. |
| Reliability | Monitoring and logging should continue correctly during normal operation. | Member 1 – monitoring; Member 4 – logging | All module owners test error and recovery paths. |
| Maintainability | Code shall be modular and documented sufficiently for team maintenance. | All four members | Each member maintains documentation for their module; shared conventions apply. |
| Testability | Functional components shall be independently testable where practical. | All four members | Each member writes unit tests; the team performs integration/system tests. |

| | | | |
|---|---|---|---|
| Portability | OS-specific operations should be isolated to facilitate future support for other platforms. | Member 1 – monitoring interfaces; Member 3 – optimization interfaces | Other members keep OS-specific assumptions out of shared logic. |

### 6.5 Business Rules

| Business Rule | Requirement Owner |
|---|---|
| The user controls whether monitoring is active. | Member 1 – Ankita Vellara |
| Optimization shall respect the selected performance profile. | Member 3 – Harshitha Vangaveti |
| User confirmation is required before potentially disruptive actions. | Member 3 – Harshitha Vangaveti |
| System-critical processes are protected from unsafe optimization. | Member 3 – Harshitha Vangaveti |
| Implemented functionality shall be traceable to requirements and test cases. | All four members; coordinated through the project repository. |

## 7. Other Requirements

The system shall support project documentation and traceability required by the JACKFRUIT Software Engineering course. Requirements, source code, tests, and changes shall be maintained in the project repository. The implementation should support future extension of detection rules, profiles, monitoring metrics, and optimization strategies.

The team shall use Git and GitHub for version control and collaboration, a GitHub Project or equivalent Agile planning tool for backlog and sprint tracking, automated testing/CI checks, code review, and suitable static-analysis/security validation. The exact deployment/packaging approach shall be selected to suit the Windows desktop application and its host-operating-system access requirements.

## Appendix A: Glossary

| Term | Definition |
|---|---|
| Performance Throttling | Reduction in application/system performance caused by resource, thermal, process, power, or related constraints. |
| CPU | Central Processing Unit. |
| RAM | Random Access Memory. |
| Process | A running program instance managed by the operating system. |

| | |
|---|---|
| Profile | A user-selected set of performance and optimization settings. |
| Monitoring | Continuous or periodic collection and display of system/application metrics. |
| NFR | Non-Functional Requirement. |
| PR | Pull Request; a proposed change submitted for review before merging. |

## Appendix B: Field Layouts

The following logical fields represent information that may be stored/displayed by the application. Exact implementation types may be refined during design.

| Field | Data Type | Mandatory | Description | Related Owner |
|---|---|---|---|---|
| Timestamp | DateTime | Y | Time at which a metric/event was recorded. | Members 1, 2 and 4 |
| CPU Usage | Decimal | Y | CPU utilization percentage. | Member 1 |
| Memory Usage | Decimal | Y | Memory utilization percentage. | Member 1 |
| Process Name | String | Y | Name of the monitored process. | Members 1 and 3 |
| Process CPU Usage | Decimal | N | CPU usage attributed to a process. | Members 1 and 3 |
| Process Memory Usage | Decimal | N | Memory usage attributed to a process. | Members 1 and 3 |
| Network Usage | Decimal | N | Relevant network usage metric. | Member 4 |
| Profile Name | String | Y | Name of the active performance profile. | Member 3 |
| Event Type | String | Y | Type of detected condition or action. | Members 2 and 4 |
| Action Result | String | N | Outcome of an optimization action. | Members 3 and 4 |

Report requirements: The monitoring/report view should include timestamp, current resource metrics, detected events, active profile, recommended/selected action, and action result where applicable.

## Appendix C: Requirement Traceability Matrix

| Requirement ID | Brief Description | Owner | Architecture Ref. | Design Ref. | Code/File Ref. | Test Case ID |
|---|---|---|---|---|---|---|
| REQ-01 | allow the user to start and stop real-time monitoring. | Member 1 – Ankita Vellara | To be completed during design | To be completed during design | To be completed after implementation | To be completed during testing |
| REQ-02 | display current CPU and memory utilization. | Member 1 – Ankita Vellara | To be completed during design | To be completed during design | To be completed after implementation | To be completed during testing |
| REQ-03 | display relevant process-level resource usage. | Member 1 – Ankita Vellara | To be completed during design | To be completed during design | To be completed after implementation | To be completed during testing |
| REQ-04 | refresh monitoring information automatically at a defined interval. | Member 1 – Ankita Vellara | To be completed during design | To be completed during design | To be completed after implementation | To be completed during testing |
| REQ-05 | evaluate monitored metrics against configured detection criteria. | Member 2 – Ashritha Reddy | To be completed during design | To be completed during design | To be completed after implementation | To be completed during testing |
| REQ-06 | identify and report suspected performance-throttling conditions. | Member 2 – Ashritha Reddy | To be completed during design | To be completed during design | To be completed after implementation | To be completed during testing |
| REQ-07 | record the time and relevant metrics associated with a detected event. | Member 2 – Ashritha Reddy | To be completed during design | To be completed during design | To be completed after implementation | To be completed during testing |

| | | | | | | |
|---|---|---|---|---|---|---|
| REQ-08 | avoid reporting a condition as certain when the available evidence only indicates a suspected event. | Member 2 – Ashritha Reddy | To be completed during design | To be completed during design | To be completed after implementation | To be completed during testing |
| REQ-09 | list relevant running processes and their resource usage. | Member 3 – Harshitha Vangaveti | To be completed during design | To be completed during design | To be completed after implementation | To be completed during testing |
| REQ-10 | identify eligible background processes that may affect performance. | Member 3 – Harshitha Vangaveti | To be completed during design | To be completed during design | To be completed after implementation | To be completed during testing |
| REQ-11 | require user confirmation before disruptive optimization actions. | Member 3 – Harshitha Vangaveti | To be completed during design | To be completed during design | To be completed after implementation | To be completed during testing |
| REQ-12 | report the result of each optimization action. | Member 3 – Harshitha Vangaveti | To be completed during design | To be completed during design | To be completed after implementation | To be completed during testing |
| REQ-13 | display relevant network usage statistics. | Member 4 – Hamsa M N | To be completed during design | To be completed during design | To be completed after implementation | To be completed during testing |
| REQ-14 | associate network usage information with the monitoring period. | Member 4 – Hamsa M N | To be completed during design | To be completed during design | To be completed after implementation | To be completed during testing |

| | | | | | | |
|---|---|---|---|---|---|---|
| REQ-15 | alert the user when configured network-usage conditions are exceeded. | Member 4 – Hamsa M N | To be completed during design | To be completed during design | To be completed after implementation | To be completed during testing |
| REQ-16 | provide predefined performance profiles. | Member 3 – Harshitha Vangaveti | To be completed during design | To be completed during design | To be completed after implementation | To be completed during testing |
| REQ-17 | allow users to select an active profile. | Member 3 – Harshitha Vangaveti | To be completed during design | To be completed during design | To be completed after implementation | To be completed during testing |
| REQ-18 | allow supported profile settings to be modified. | Member 3 – Harshitha Vangaveti | To be completed during design | To be completed during design | To be completed after implementation | To be completed during testing |
| REQ-19 | apply the active profile to relevant optimization decisions. | Member 3 – Harshitha Vangaveti | To be completed during design | To be completed during design | To be completed after implementation | To be completed during testing |
| REQ-20 | display alerts for relevant detected conditions. | Member 2 – Ashritha Reddy | To be completed during design | To be completed during design | To be completed after implementation | To be completed during testing |
| REQ-21 | provide optimization recommendations when sufficient information is available. | Member 2 – Ashritha Reddy | To be completed during design | To be completed during design | To be completed after implementation | To be completed during testing |
| REQ-22 | log detected events and optimization actions with timestamps. | Member 4 – Hamsa M N | To be completed during design | To be completed during design | To be completed after implementation | To be completed during testing |

| | | | | | | |
|---|---|---|---|---|---|---|
| REQ-23 | allow the user to view recent logs. | Member 4 – Hamsa M N | To be completed during design | To be completed during design | To be completed after implementation | To be completed during testing |

## Appendix D: Team Responsibility Matrix

| Member | Name | Owned Functional Requirements | Owned NFR Areas |
|---|---|---|---|
| Member 1 | Ankita Vellara | REQ-01 to REQ-04 | Dashboard update frequency, monitoring overhead, responsiveness, permitted monitoring data, monitoring-side portability. |
| Member 2 | Ashritha Reddy | REQ-05 to REQ-08; REQ-20 to REQ-21 | Detection correctness and cautious reporting; alert usability; contribution to system responsiveness and error communication. |
| Member 3 | Harshitha Vangaveti | REQ-09 to REQ-12; REQ-16 to REQ-19 | Optimization responsiveness, safety, user confirmation, safe failure, OS permissions, optimization-side portability. |
| Member 4 | Hamsa M N | REQ-13 to REQ-15; REQ-22 to REQ-23 | Network privacy, log data minimization, logging reliability, network monitoring security. |

Shared responsibilities: All members shall contribute implementation commits, code reviews, testing, documentation, integration, and requirement traceability. Primary ownership identifies the member responsible for coordinating implementation of a requirement.